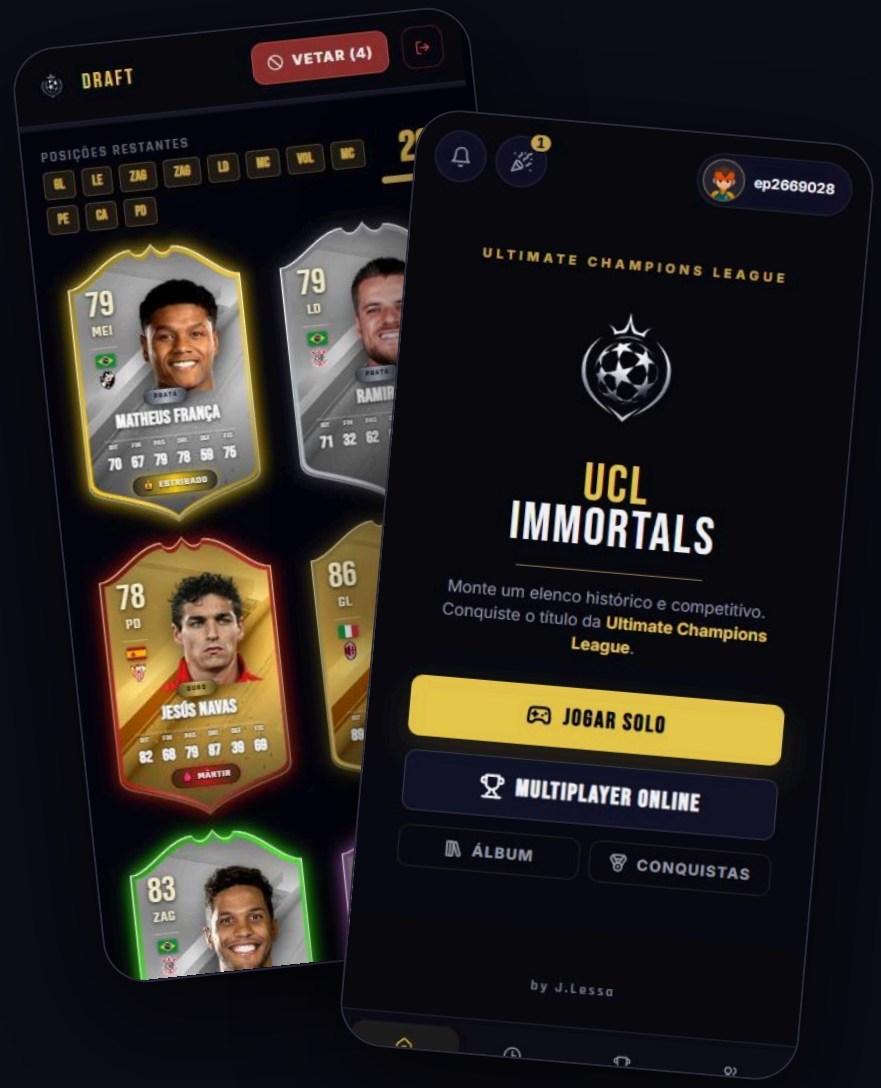
PROJETO FINAL · OPÇÃO 1

UCL Immortals na AWS

Proposta de infraestrutura para um jogo online de futebol

Desenvolvimento de Aplicações Distribuídas

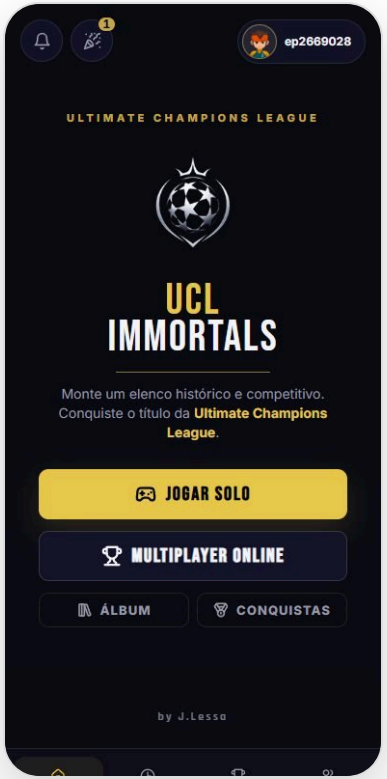
José Lessa · outubro de 2026



1 · A APLICAÇÃO

O jogo

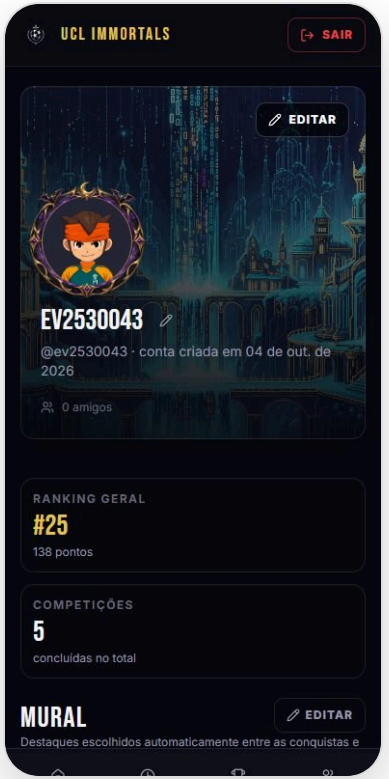
- Monta um time histórico **por draft**, uma carta por vez
- Disputa uma competição no formato da **Liga dos Campeões**
- Partidas **simuladas pelo servidor** a partir de atributos, química, técnico e tática
- **Solo** contra o computador ou **online** com até 8 pessoas na mesma sala
- **Contas** com histórico, ranking, conquistas, eventos e amigos



Início



Draft



Perfil

O servidor é a autoridade

O navegador só envia ações. O servidor valida, aplica as regras e devolve o mesmo estado para todos da sala.

Estado compartilhado

Uma sala, uma verdade

Até 8 jogadores veem a mesma competição, os mesmos resultados e as mesmas negociações.

Tempo real

Ações sincronizadas

Draft, rodadas, loja, mercado e trocas chegam a todos por WebSocket, na hora.

Dados persistentes

Nada se perde

Pontos, recordes e conquistas são calculados e gravados no servidor, nunca no navegador.

Uma segunda implantação, na AWS

O jogo já está no ar. O projeto cria uma versão completa e independente na AWS, a partir de um ramo separado do código.

Versão atual

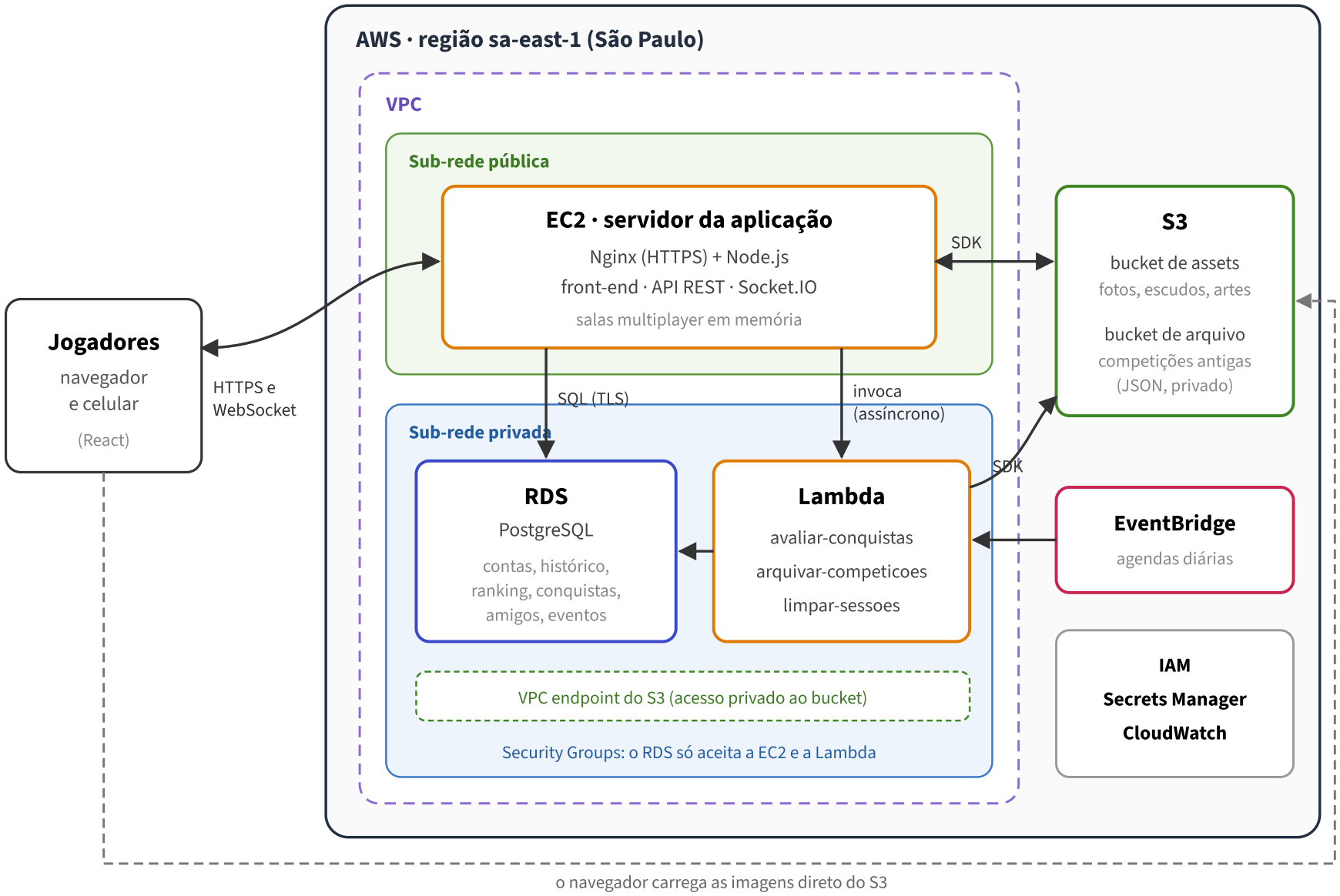
Hospedagem própria, ramo principal

Continua no ar, sem nenhuma alteração.

Versão da disciplina

AWS, ramo próprio, banco próprio

Servidor em VM, banco gerenciado, armazenamento de objetos e funções serverless.



Setas contínuas: chamadas entre componentes · tracejada: imagens carregadas direto do S3

4 · COMPONENTES

EC2 · servidor da aplicação

Amazon Linux 2023 · t3.small · Elastic IP · Nginx (HTTPS) + Node.js com systemd

Front-end

Entrega a aplicação React compilada. As imagens não passam por aqui: vêm do S3.

API REST · 16 rotas

Cadastro e login, perfil, histórico, ranking e recordes, conquistas, eventos, amigos, convites e curtidas.

Tempo real · Socket.IO

Salas, draft, rodadas, loja, mercado e trocas, com cerca de 30 tipos de mensagem. A sala vive em memória durante a competição.

4 · COMPONENTES

RDS · PostgreSQL

Banco gerenciado · db.t3.micro · sub-rede privada · backup diário (7 dias) · conexão com TLS · migrações versionadas

Área	Tabelas	O que guarda
Contas	`users` `sessions` `auth_rate_limits`	Usuários, senhas com hash, sessões, limite de login
Perfil	`profiles` `profile_stats` `profile_likes`	Aparência, mural, estatísticas, curtidas
Competições	`competition_history` `competition_records`	Cada competição concluída e os recordes
Progressão	`user_achievements` `user_frames`	Conquistas e molduras de evento
Social	`friendships` `room_invitations` `account_presence`	Amizades, convites e quem está online

4 · COMPONENTES

S3 · dois buckets, via SDK

Leitura pública

Bucket de assets

Todos os arquivos estáticos do jogo: **~1.900 arquivos, 44 MB**. Fotos de jogadores, escudos, técnicos, estádios, cartas, avatares, capas e artes.

Enviados por um script de implantação com o SDK; o navegador baixa direto do S3.

Privado

Bucket de arquivo

O retrato completo de cada competição (time final e jogos), em JSON, depois de **30 dias**.

Ao abrir uma competição antiga no histórico, o servidor busca o arquivo com o SDK.

Lambda + EventBridge · serverless

Funções em Node.js, na mesma VPC do banco, para o trabalho que não precisa acontecer enquanto o jogador espera.

Função	Disparo	O que faz
avaliar-conquistas	EC2, assíncrono, ao salvar uma competição	Recalcula as 40 conquistas e os eventos; grava níveis e molduras
arquivar-competicoes	EventBridge, toda madrugada	Move para o S3 os retratos com mais de 30 dias
limpar-sessoes	EventBridge, todo dia	Apaga sessões vencidas e registros antigos

Só a EC2 fica exposta

Security Group	Entrada permitida
sg-aplicacao	80 e 443 de qualquer lugar; SSH só do administrador
sg-banco	5432 só da aplicação e das funções
sg-funcoes	Nenhuma

- **Sub-redes privadas** para RDS e Lambda, sem rota para a internet
- **VPC endpoint**: as funções chegam ao S3 sem sair da rede da AWS
- **IAM**: cada componente com só as permissões de que precisa
- **Secrets Manager**: senha do banco fora do código
- **CloudWatch**: logs, métricas e alarmes

7 · FLUXO PRINCIPAL

Quando uma competição termina

1. O servidor na **EC2** recalcula fase e pontos (não confia no navegador) e grava no **RDS**.
2. Invoca **avaliar-conquistas** sem esperar e responde ao jogador na hora.
3. A **Lambda** lê o histórico, recalcula conquistas e eventos e grava o que foi desbloqueado.
4. 30 dias depois, o **EventBridge** dispara o arquivamento e o retrato vai para o **S3**.

No online é igual: o servidor grava o resultado de cada jogador com conta da sala e invoca a função para cada um.

O que é pedido e como é atendido

Requisito	Como é atendido
Instância com a aplicação	EC2 com Nginx e Node.js: front-end, API e tempo real
SGBD separado	RDS com PostgreSQL (alternativa aceita à segunda instância)
Blob via SDK	S3 com os assets do jogo e o arquivo de competições
Serviço serverless	Três funções Lambda, duas agendadas pelo EventBridge
Recursos adicionais	VPC e Security Groups, VPC endpoint, IAM, Secrets Manager, CloudWatch

Onde a proposta para, e para onde iria

Limitação

As salas online vivem na memória de **uma única instância**. Se ela reiniciar, as salas em andamento se perdem; os dados das contas, no RDS, não.

Evolução

Load Balancer + Auto Scaling com várias instâncias, estado das salas no **ElastiCache (Redis)**, que também liga as instâncias do Socket.IO, e **CloudFront** na frente do bucket de assets.